

A
Summer Internship Report
On
"E-Commerce Web Application"
"Social-Media Web Application"

(CSE402 – Summer Internship - II)

Prepared by
Harsh Pawar(22CS069)

Under the Supervision of
Prof. Brinda Patel

Submitted to
Charotar University of Science & Technology (CHARUSAT)
for the Partial Fulfillment of the Requirements for the
Degree of Bachelor of Technology (B.Tech.)
for Semester 7

Submitted at



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Chandubhai S. Patel Institute of Technology (CSPIT)
Faculty of Technology & Engineering (FTE), CHARUSAT
At: Changa, Dist: Anand, Pin: 388421.
2025-26



CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Accredited with Grade A+ by NAAC

CERTIFICATE

This is to certify that the report entitled “**E-Commerce Web Application and Social-media Web Application**” is a bonafied work carried out by **Harsh Pawar(22CS069)** under the guidance and supervision of **Prof. Brinda Patel / Mrs. Swati Srivastava** for the subject **Summer Internship – II (CSE402)** of 7th Semester of Bachelor of Technology in **Computer Science & Engineering** at Chandubhai S. Patel Institute of Technology (CSPIT), Faculty of Technology & Engineering (FTE) – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred by the examiner(s).

Under the supervision of,

Prof. Brinda Patel

Assistant Professor

Dept. of Computer Science & Engineering
CSPIT, FTE, CHARUSAT, Changa, Gujarat

Ms. Swati Srivastava

Founder & CEO

Code Alpha

Dr. Amit Thakkar

Head - Department of Computer Science & Engineering,
CSPIT, FTE, CHARUSAT, Changa, Gujarat.

Chandubhai S. Patel Institute of Technology (CSPIT)
Faculty of Technology & Engineering (FTE), CHARUSAT

At: Changa, Ta. Petlad, Dist. Anand, Pin: 388421. Gujarat.

Date: 30/06/2026

SUMMER INTERNSHIP COMPLETION CERTIFICATE

This is to certify that **Harsh Pawar**, B.Tech (CSE) students of Chandubhai S. Patel Institute of Technology, CHARUSAT, Changa has done a summer internship on “**E-Commerce Web Application & Social-Media Web Application**” at **Code Alpha** from **01/06/2025** to **30/06/2026**.

We wish him all the best for his bright career.



Thanking you.

For Code Alpha

Ms. Swati Srivastava

Founder & CEO

TABLE OF CONTENTS

ABSTRACT.....	i
ACKNOWLEDGEMENT	ii
DESCRIPTION OF COMPANY / ORGANIZATION	iii
CHAPTER 1 INTRODUCTION	4
1.1 PURPOSE OF INTERNSHIP	5
1.2 OVERVIEW OF PROJECT	5
1.3 OBJECTIVE	6
1.4 SCOPE	6
1.5 ROLES AND RESPONSIBILITIES	6
1.6 INTERNSHIP PLAN (WEEK WISE)	7
CHAPTER 2 SYSTEM ANALYSIS	9
2.1 STUDY OF EXISTING SYSTEMA AND LIMITATIONS	10
2.2 REQUIREMENT OF NEW SYSTEM	10
2.2.1 Functional Requirements:	10
2.2.2 Non-functional Requirements:	11
2.3 HARDWARE REQUIREMENT	12
2.4 SOFTWARE REQUIREMENT	12
CHAPTER 3 DEVELOPMENT ENVIRONMENT	13
3.1 DEVELOPMENT ENVIROMENT.....	14
CHAPTER 4 SYSTEM DESIGN	15
4.1 DATA DICTIONARY	16
4.1.1 E-Commerce Web Application	16
4.1.2 Social-Media Web Application	18
4.2 ER DIAGRAM	21
4.2.1 E-Commerce Web Applications.....	21
4.2.1 Social-Media Web Application	22

4.3 CLASS DIAGRAM	23
4.3.1 E-Commerce Web Application	23
4.3.1 Social-Media Web Application	24
CHAPTER 5 IMPLEMENTATION SCREENSHOTS	25
5.1 SCREENSHOTS.....	26
5.1.1 E-Commerce Web Application Screenshots	26
5.1.2 Social-Media Web Application Screenshots	30
CHAPTER 6 TEST CASES	35
6.1 e-commerce web application test cases	36
6.2 social-media web application test cases.....	39
CHAPTER 7 LIMITATIONS AND FUTURE WORK	43
7.1 E-COMMERCE WEB APPLICATION	44
7.1.1 Limitations	44
7.1.2 Future Work	44
7.2 SOCIAL-MEDIA WEB APPLICATION	45
7.2.1 Limitations	45
7.2.2 Future Work	46
CHAPTER 8 CONCLUSION.....	47
8.1 CONCLUSION.....	48
REFERENCES	49

LIST OF TABLES

Table 1. 1 Internship Weekly Planning	7
Table 4. 1 E-Commerce Web Application Data Dictionary	16
Table 4. 2 Social-Media Web Application Data Dictionary	18
Table 6. 1 E-Commerce Web Application Test Cases	36
Table 6. 2 Social-Media Web Application Test Cases	39

LIST OF FIGURES

Fig 4.1 E-Commerce Web Application ER-Diagram.....	21
Fig 4.2 Social-Media Web Application ER-Diagram	22
Fig 4.3 E-Commerce Web Application Class-Diagram	23
Fig 4.4 Social-Media Web Application Class-Diagram	24
Fig 5.1 SignUp Interface.....	26
Fig 5.2 Login Interface	26
Fig 5.3 Home Interface	27
Fig 5.4 Product Interface.....	27
Fig 5.5 Checkout Interface.....	28
Fig 5.6 Order Summary Interface	28
Fig 5.7 Order Completion Interface.....	29
Fig 5.8 Order History Interface.....	29
Fig 5.9 Profile Interface	30
Fig 5.10 login Interface.....	30
Fig 5.11 SignUp Interface.....	31
Fig 5.12 User Profile Interface.....	31
Fig 5.13 Create Post Interface.....	32
Fig 5.14 Upload Reel Interface	32
Fig 5.15 Chat Interface	33
Fig 5.16 Reels Feed Interface	33
Fig 5.17 Comment Interface	34

ABSTRACT

This report summarizes the development of two full-stack web applications completed during a 4-week remote internship at CodeAlpha from June 1 to June 30, 2026. The internship focused on hands-on experience with the MERN stack (MongoDB, Express.js, React, Node.js) through two projects: an E-commerce platform and a Social Media web app inspired by Instagram. The E-commerce application, built in the first two weeks, includes features like user authentication (JWT, bcryptjs), product management with CRUD operations, a shopping cart, order system, admin panel, reviews, wishlists, and address handling. The frontend uses React, Redux Toolkit, and Material-UI for a smooth user experience, while the backend handles secure and scalable business logic with Express.js and MongoDB. The Social Media app, developed in the final two weeks, offers user profiles, real-time follow/unfollow, post creation with likes and comments, chat via Socket.io, reels upload, and image handling with Cloudinary and Multer. It incorporates advanced state management and real-time features for better engagement. Both projects followed security best practices, used responsive design, form validation, and RESTful APIs, and tackled challenges like token management, media handling, and performance optimization. Overall, the internship provided valuable practical experience in full-stack development and real-world project architecture, greatly enhancing both technical and problem-solving skills.

ACKNOWLEDGEMENT

The successful completion of this internship and the preparation of this final report would not have been possible without the guidance, support, and encouragement of many individuals and organizations. First and foremost, I express my deepest gratitude to **CHARUSAT, Chandubhai S. Patel Institute of Technology**, for providing the opportunity and resources to undertake this remote internship at **CodeAlpha**. Their academic mentorship and administrative support have been invaluable throughout this journey. I am sincerely thankful to my internal guide **Assist. Prof. Brinda Patel**, and external guide **Ms. Swati Srivastava**, whose expert advice, constructive feedback, and timely suggestions greatly enhanced the quality of my work. Her willingness to clarify doubts and share best practices in full-stack development motivated me to overcome technical challenges and refine my solutions. I would also like to acknowledge my external guide at CodeAlpha, **Ms. Swati Srivastava**, for entrusting me with meaningful project tasks, guiding me through real-world software development workflows, and encouraging me to adopt industry-standard tools and methodologies. Her insights into authentication security, state management patterns, and real-time communication protocols were instrumental in the successful delivery of both the e-commerce and social media applications. Special thanks go to my peers and colleagues at CodeAlpha for fostering a collaborative environment, sharing knowledge, and providing moral support during debugging sessions and code reviews. Their camaraderie made remote teamwork both productive and enjoyable.

DESCRIPTION OF COMPANY / ORGANIZATION

CodeAlpha is an Indian-based software services and EdTech platform founded around 2022–2023. The company operates primarily through remote and virtual internship programs covering domains like web development, app development, Python programming, and graphic design. Marketed as a platform offering one-month remote internships, CodeAlpha emphasizes hands-on project-based learning aligned with modern technologies (e.g., React, Java, Python). They claim hundreds of thousands of learners and co-branded certificates in partnership with institutions like Microsoft and IIT (though these are not official educational approvals). Beyond internships, the company promotes itself as offering software and graphic design services—including website design, mobile apps, packaging, and branding—serving national and international clients.

CHAPTER 1
INTRODUCTION

1.1 PURPOSE OF INTERNSHIP

The primary objective of this internship was to bridge the gap between academic knowledge and real-world software development by engaging in the end-to-end creation of two full-stack web applications. Conducted over four weeks at CodeAlpha, the internship was designed to offer practical exposure to the MERN stack (MongoDB, Express.js, React, and Node.js), enabling a deeper understanding of how modern web applications are built and deployed. Through this experience, I reinforced core development practices such as secure user authentication, efficient state management, and RESTful API design. Additionally, the internship strengthened my frontend development skills by working with React, Redux Toolkit, and Material-UI to build responsive and user-friendly interfaces. I also gained familiarity with professional workflows, including the use of version control systems, participation in code reviews, and adherence to agile development practices. By addressing real-world challenges like debugging asynchronous behavior, handling real-time communication, and optimizing application performance, this internship significantly enhanced my problem-solving abilities and prepared me for a career in full-stack development.

1.2 OVERVIEW OF PROJECT

During this internship, I developed two full-stack web applications using the MERN stack. The first, an E-commerce platform built in Weeks 1–2, featured secure user authentication (JWT, bcryptjs) with OTP-based password reset, product and category management, shopping cart and order functionality, wishlists, reviews, and a role-based admin panel. The frontend was built using React, Redux Toolkit, React Hook Form, and Material-UI, offering a responsive and user-friendly experience.

In Weeks 3–4, I developed a Social Media Web Application inspired by Instagram. It included user signup/login, profile management, a real-time follow/unfollow system, post creation with likes and comments, media uploads (images via Cloudinary, reels via Multer), and real-time messaging using Socket.io. Redux Toolkit managed frontend state, ensuring seamless interaction across features.

1.3 OBJECTIVE

The main goal of this internship was to gain hands-on experience in full-stack development by building secure and scalable web applications. I focused on implementing authentication with token management and protected routes, designing RESTful APIs, and creating responsive user interfaces using React, Redux Toolkit, and Material-UI. I also integrated real-time features like chat using Socket.io and handled media uploads with Cloudinary and Multer. Throughout the process, I followed best practices to ensure the code was clean, modular, and ready for deployment.

1.4 SCOPE

This four-week internship covered the complete software development lifecycle for two full-stack applications. It began with gathering requirements and choosing the appropriate tech stack, followed by designing the backend architecture and database models using MongoDB and Mongoose. I developed APIs in Express.js, handling authentication, authorization, and core CRUD operations. On the frontend, I built responsive interfaces with React, managed state using Redux Toolkit, and styled components with Material-UI. The projects also involved integrating third-party services like Cloudinary for image handling and email-based OTP verification, as well as implementing real-time features such as chat using Socket.io. Finally, I focused on testing, debugging, and preparing the applications for deployment.

1.5 ROLES AND RESPONSIBILITIES

During the internship, I handled both backend and frontend development across two full-stack projects. On the backend, I set up the Express.js server, designed MongoDB schemas, and implemented features like JWT authentication, email-based OTP verification, password reset, and RESTful APIs. On the frontend, I built React components for user authentication, product listings, profiles, post feeds, reels, and chat interfaces, while managing application state using Redux Toolkit and connecting APIs with Axios. I also integrated real-time functionality using Socket.io for chat and live follower updates. To ensure a smooth user experience, I designed responsive UIs with Material-UI, used React Hook Form for validation, and added animations using Framer Motion.

1.6 INTERNSHIP PLAN (WEEK WISE)

Table 1. 1 Internship Weekly Planning

	Date	Day	Name of Module
Week 1	02/06/2025	Monday	Set up backend infrastructure (Express.js server, MongoDB connection) Implemented user login/signup with bcryptjs
	03/06/2025	Tuesday	Added JWT-based authentication middleware.
	04/06/2025	Wednesday	Integrated email-based OTP verification
	05/06/2025	Thursday	Built secure password-reset flow
	06/06/2025	Friday	Developed React authentication screens (Redux Toolkit, MUI)
	07/06/2025	Saturday	Added form validation (react-hook-form) and API calls (Axios, Thunks)
Week 2	09/06/2025	Monday	Created product CRUD APIs
	10/06/2025	Tuesday	Built shopping cart system APIs
	11/06/2025	Wednesday	Developed wishlist & reviews endpoints
	12/06/2025	Thursday	Implemented address management & order APIs. Added role-based access control & admin panel
	13/06/2025	Friday	Set up protected routes and role-based navigation
	14/06/2025	Saturday	Styled responsive product and cart pages (React Router, MUI)
Week 3	16/06/2025	Monday	Implemented social-app JWT authentication
	17/06/2025	Tuesday	Built follow/unfollow system with real-time count. Created user search & profile-update APIs
	018/06/2025	Wednesday	Integrated image upload with Multer & Cloudinary
	19/06/2025	Thursday	Developed signup/login pages (frontend)
	20/06/2026	Friday	Built explore & profile pages (Redux state, route protection)
	21/06/2025	Saturday	Debugged auth edge cases (token expiry handling)

Week 4	22/06/2025	Monday	Created post-CRUD APIs
	23/06/2025	Tuesday	Implemented like/comment endpoints & notifications
	24/06/2025	Wednesday	Developed chat/message APIs (Socket.io backend)
	25/06/2025	Thursday	Built real-time chat UI with live typing indicators
	26/06/2025	Friday	Added reels upload API and storage logic
	27/06/2025	Saturday	Implemented reels playback component

CHAPTER 2
SYSTEM ANALYSIS

2.1 STUDY OF EXISTING SYSTEM AND ITS LIMITATIONS

E-commerce Web Application: In many small-scale or traditional online stores, existing systems are often built using static HTML or basic PHP/MySQL setups that lack modern features and robust security. One of the major limitations is weak authentication, where passwords are stored in plain text without hashing, making them vulnerable to breaches. Additionally, the absence of proper token management—such as session expiry or refresh mechanisms—leaves users exposed to session hijacking. Inventory management is often manual, requiring admins to directly modify database entries, which can lead to errors and inconsistencies. The user experience also suffers due to static page reloads for every action, resulting in slow and clunky shopping, cart, and checkout flows.

Social Media Web Application: Legacy social platforms or traditional discussion forums offer limited user engagement due to the lack of real-time features. Users cannot send instant messages or receive live notifications, which slows down interaction and reduces the dynamic feel of the platform. Media handling is also inefficient, with images and videos stored directly on the same server without cloud integration or CDN support, leading to high latency and poor performance. Furthermore, these systems are often built on a monolithic architecture where frontend and backend are tightly coupled, making it difficult to scale or update individual features independently. Basic user interactions such as following or liking posts often require full page reloads, which diminishes user engagement and overall experience.

2.2 REQUIREMENT OF NEW SYSTEM

2.2.1 Functional Requirements:

E-commerce Application:

1. Secure User Authentication:

- Password hashing with bcryptjs
- JWT-based tokens for session management
- Email-based OTP verification and password reset

2. Product Management:

- CRUD operations for products, brands, categories
- Role-based admin panel for product creation and updates

3. Shopping Cart & Order Processing:

- Add, remove, update item quantities
 - Place orders, view order history
 - Address management and checkout workflow
4. **Wishlist & Reviews:**
 - Add products to wishlist
 - Submit product reviews with star ratings
 5. **Responsive UI:**
 - Dynamic filters and protected routes
 - Real-time feedback on cart operations

Social Media Web Application:

1. **User Profiles & Authentication:**
 - JWT-based signup/login, secure profile updates
2. **Follow/Unfollow System:**
 - Manage followers and following lists with live count updates
3. **Post CRUD & Interactions:**
 - Create, edit, delete posts
 - Like and comment functionality with notifications
4. **Real-Time Chat:**
 - One-to-one messaging via Socket.io
 - Live typing indicators and unread message badges
5. **Media Upload & Reels:**
 - Image uploads with Cloudinary and Multer
 - Short video uploads and playback component
6. **Search & Explore:**
 - User search API
 - Explore feed of recent posts and reels

2.2.2 Non-functional Requirements:

- **Performance:** Page load time under 2 seconds for authenticated views; real-time messages delivered with latency under 200ms.
- **Scalability:** Modular microservice-friendly code structure; ability to add new modules without downtime.

- **Security:** HTTPS enforced; CORS and SameSite cookie policies configured; input validation to prevent XSS and SQL/NoSQL injection.
- **Reliability:** 99.9% uptime target; error logging and retry mechanisms for failed API calls.
- **Maintainability:** Clean code with linting rules; comprehensive documentation and automated tests covering core workflows.
- **Usability:** Responsive design across desktop, tablet, and mobile; accessible navigation and ARIA attributes for screen readers.

2.3 HARDWARE REQUIREMENT

- Processor: Intel i5 or equivalent (4 cores minimum)
- RAM: 8 GB (16 GB recommended for development with Docker and local databases)
- Storage: 256 GB SSD (for hosting code, local MongoDB instance, and media caching)
- Network: Broadband internet with at least 20 Mbps for cloud API access and real-time data exchange

2.4 SOFTWARE REQUIREMENT

- Operating System: The user's computer or device must have a compatible operating system to run the website
- Web Browser: The website must be compatible with commonly used web browsers such as Google Chrome, Mozilla Firefox, and Safari.

CHAPTER 3

DEVELOPMENT ENVIRONMENT

3.1 DEVELOPMENT ENVIROMENT

The development of both web applications during the internship was carried out on a machine equipped with an Intel Core i5-8250U processor (4 cores, 1.6 GHz base, 3.4 GHz turbo), 16 GB DDR4 RAM, and a 512 GB NVMe SSD, supported by a stable 100 Mbps broadband connection. This hardware configuration ensured smooth local builds, fast execution of Node.js servers, and efficient handling of media uploads and real-time features. The primary operating system used was Windows 11 Pro (64-bit), while Ubuntu 22.04 LTS was employed within Docker containers to simulate a production-like environment for MongoDB and the Node.js backend, reducing OS-specific inconsistencies. The software stack included Node.js v16.14.0 and npm v8.3.1 as the runtime and package manager. Backend development was powered by Express.js v4.18.1, MongoDB v5.0, Mongoose v6.3.3 for schema modeling, bcryptjs for secure password hashing, and jsonwebtoken for JWT handling. Environment configuration was managed using dotenv, while real-time communication was enabled with Socket.io v4.4.0. Media handling was done through Multer for file uploads and Cloudinary SDK v2.1.0 for cloud storage. On the frontend, React v18.0.0 was used to build the UI, with Redux Toolkit v1.8.3 for state management and React Router v6.3.0 for navigation. The interface was styled using Material-UI v5.6.1, and API calls were made using Axios v0.27.2. Form handling and validation were managed with React Hook Form v7.31.0, and animations were implemented using Framer Motion v6.2.7.

CHAPTER 4
SYSTEM DESIGN

4.1 DATA DICTIONARY

4.1.1 E-Commerce Web Application

Table 4. 1 E-Commerce Web Application Data Dictionary

Entity	Attribute	Type	Description
User	userId	ObjectId	Unique identifier
	name	String	Full name
	email	String	Unique email address
	passwordHash	String	Hashed password
	role	String	“user” or “admin”
	createdAt	Date	Account creation timestamp
Product	productId	ObjectId	Unique identifier
	name	String	Product name
	description	String	Detailed description
	price	Number	Price in currency units
	images	[String]	URLs of product images
	category	ObjectId	Reference to Category
	brand	ObjectId	Reference to Brand
Brand	brandId	ObjectId	Unique identifier
	name	String	Brand name
Category	categoryId	ObjectId	Unique identifier

Entity	Attribute	Type	Description
	name	String	Category name
Cart	cartId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	items	[{productId, qty}]	List of products and quantities
Order	orderId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	items	[{productId, qty, price}]	Products ordered and snapshot prices
	totalAmount	Number	Total order amount
	addressId	ObjectId	Reference to Address
Wishlist	wishlistId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	products	[ObjectId]	List of products added
Review	reviewId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	productId	ObjectId	Reference to Product
	rating	Number	Star rating (1–5)
	comment	String	Review text
Address	addressId	ObjectId	Unique identifier

Entity	Attribute	Type	Description
	userId	ObjectId	Reference to User
	line1, line2, city, state, zip	String	Address fields

4.1.2 Social-Media Web Application

Table 4. 2 Social-Media Web Application Data Dictionary

Entity	Attribute	Type	Description
User	userId	ObjectId	Unique identifier
	username	String	Unique handle
	email	String	Unique email
	passwordHash	String	Hashed password
	profilePicUrl	String	URL to profile image
	bio	String	User biography
	createdAt	Date	Account creation timestamp
Follow	followerId	ObjectId	Reference to User (follower)
	followingId	ObjectId	Reference to User (followed)
Post	postId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	content	String	Post text

Entity	Attribute	Type	Description
	imageUrl	String	URL of image
	createdAt	Date	Timestamp
Like	likeId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	postId	ObjectId	Reference to Post
Comment	commentId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	postId	ObjectId	Reference to Post
	text	String	Comment text
	createdAt	Date	Timestamp
Notification	notificationId	ObjectId	Unique identifier
	userId	ObjectId	Recipient
	type	String	“like”, “comment”, “follow”
	referenceId	ObjectId	e.g. postId or followerId
	read	Boolean	Read flag
Chat	chatId	ObjectId	Unique identifier
	participants	[ObjectId]	Two user references
	messages	[{senderId, text, timestamp}]	Chat history

Entity	Attribute	Type	Description
Reel	reelId	ObjectId	Unique identifier
	userId	ObjectId	Reference to User
	videoUrl	String	URL of video
	createdAt	Date	Timestamp

4.2 ER DIAGRAM

4.2.1 E-Commerce Web Applications

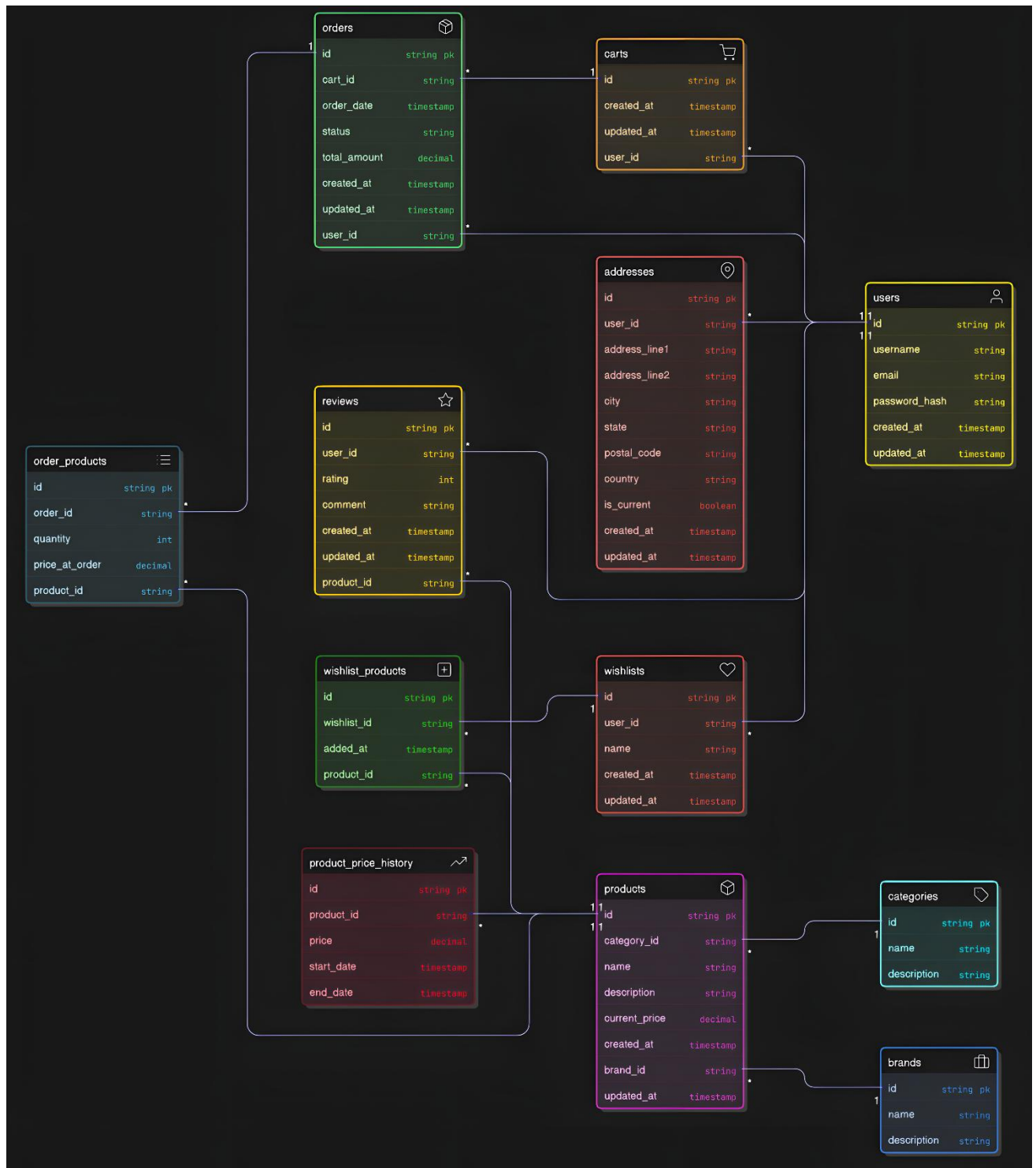


Fig 4.1 E-Commerce Web Application ER-Diagram

4.2.1 Social-Media Web Application

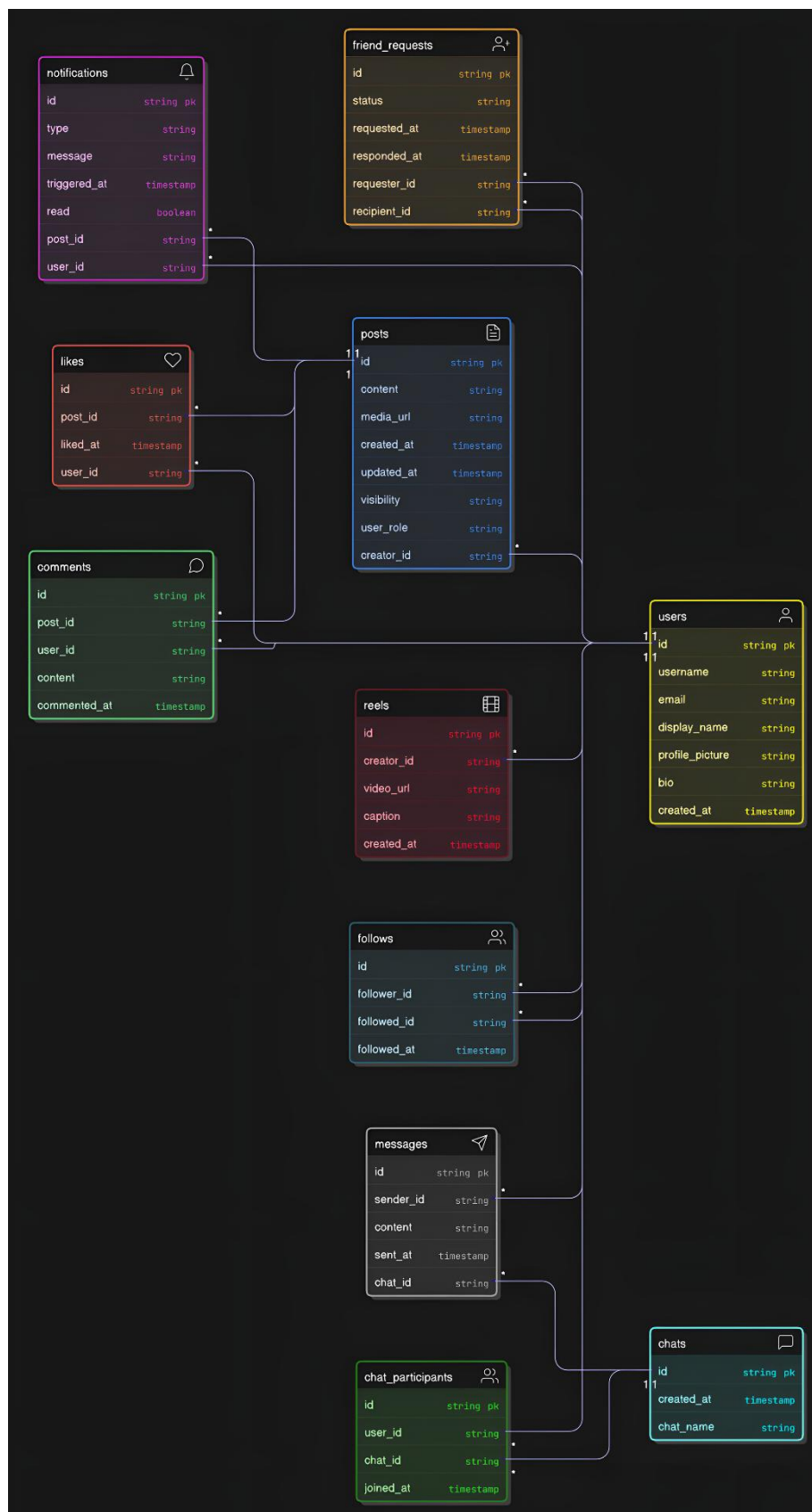


Fig 4.2 Social-Media Web Application ER-Diagram

4.3 CLASS DIAGRAM

4.3.1 E-Commerce Web Application



Fig 4.3 E-Commerce Web Application Class-Diagram

4.3.1 Social-Media Web Application

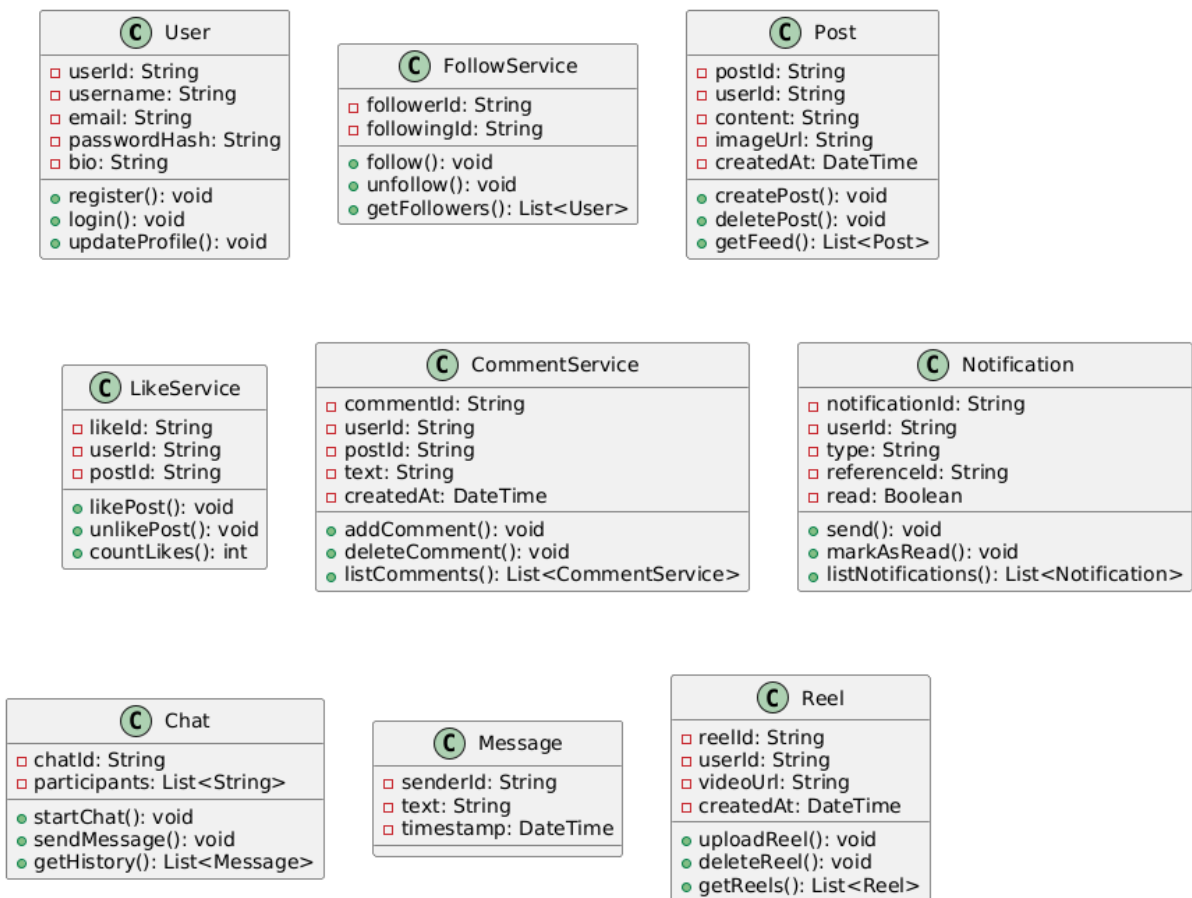


Fig 4.4 Social-Media Web Application Class-Diagram

CHAPTER 5

IMPLEMENTATION SCREENSHOTS

5.1 SCREENSHOTS

5.1.1 E-Commerce Web Application Screenshots

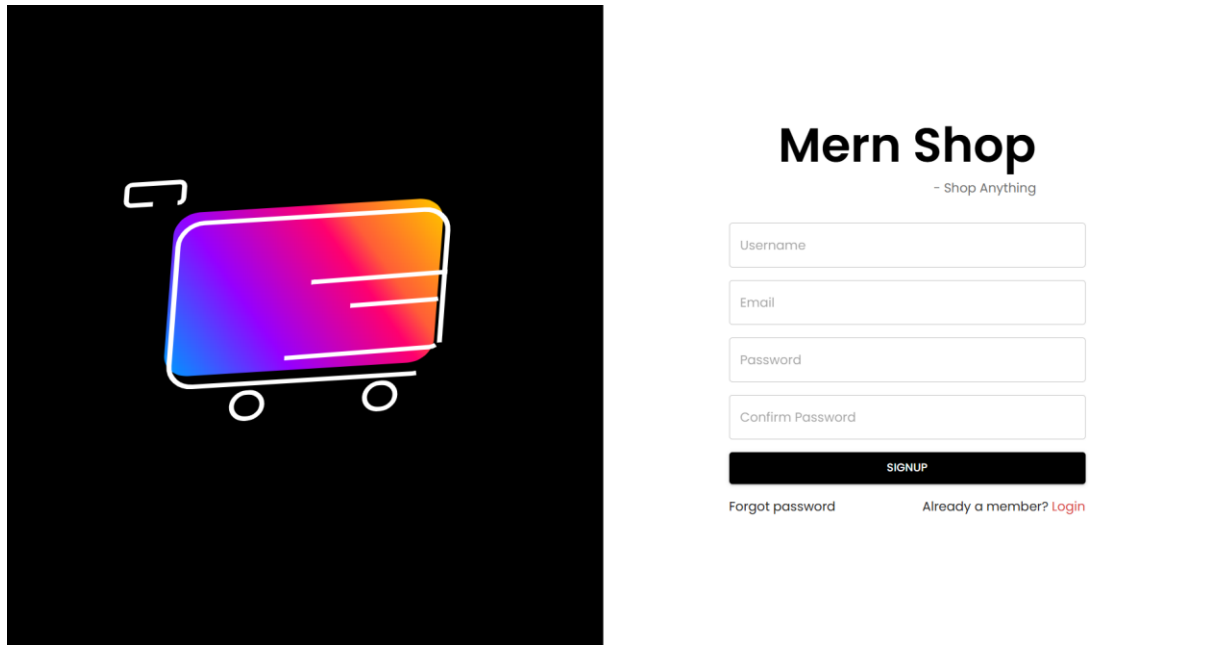


Fig 5.1 SignUp Interface

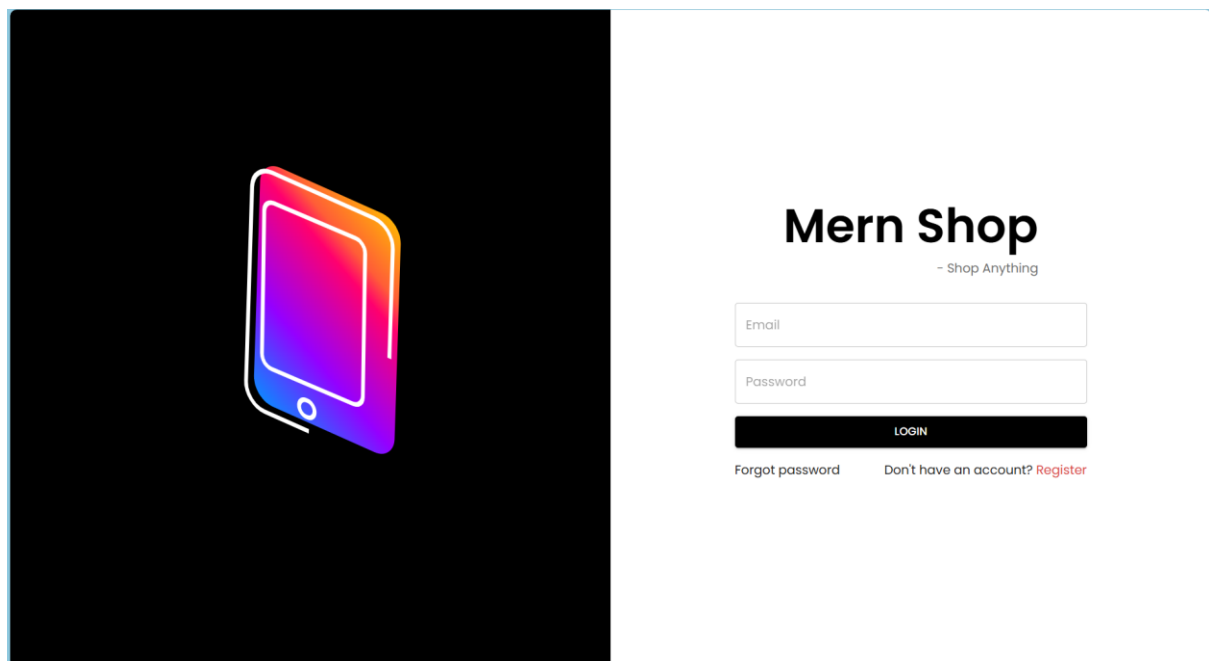


Fig 5.2 Login Interface

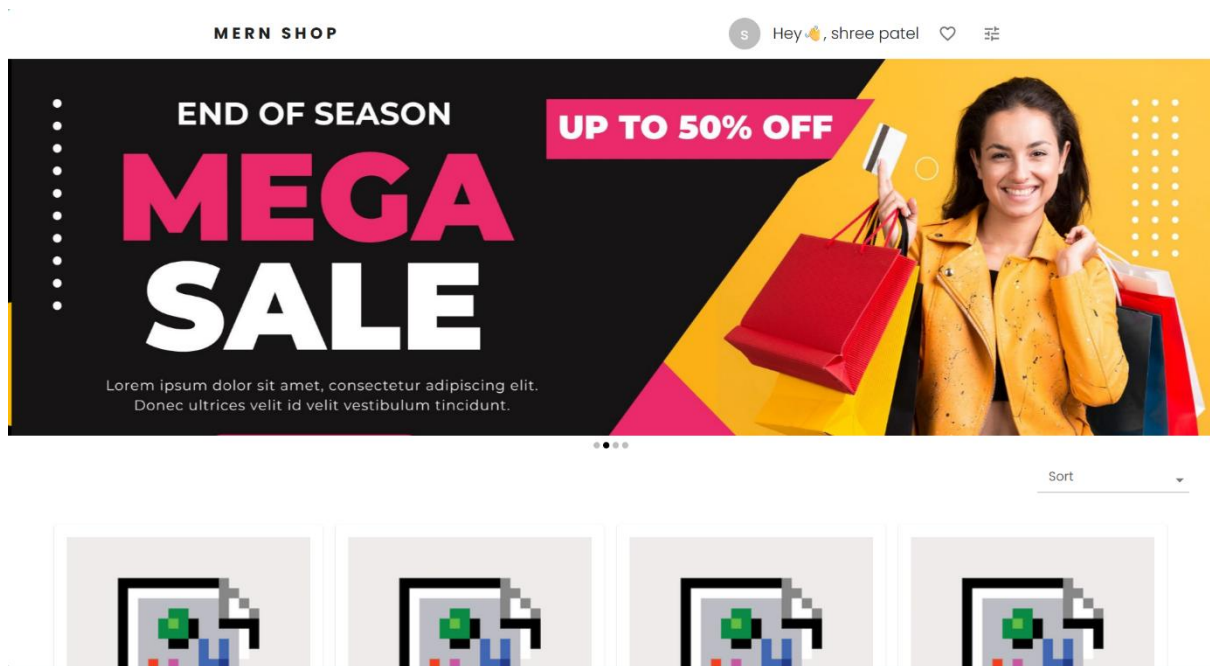


Fig 5.3 Home Interface

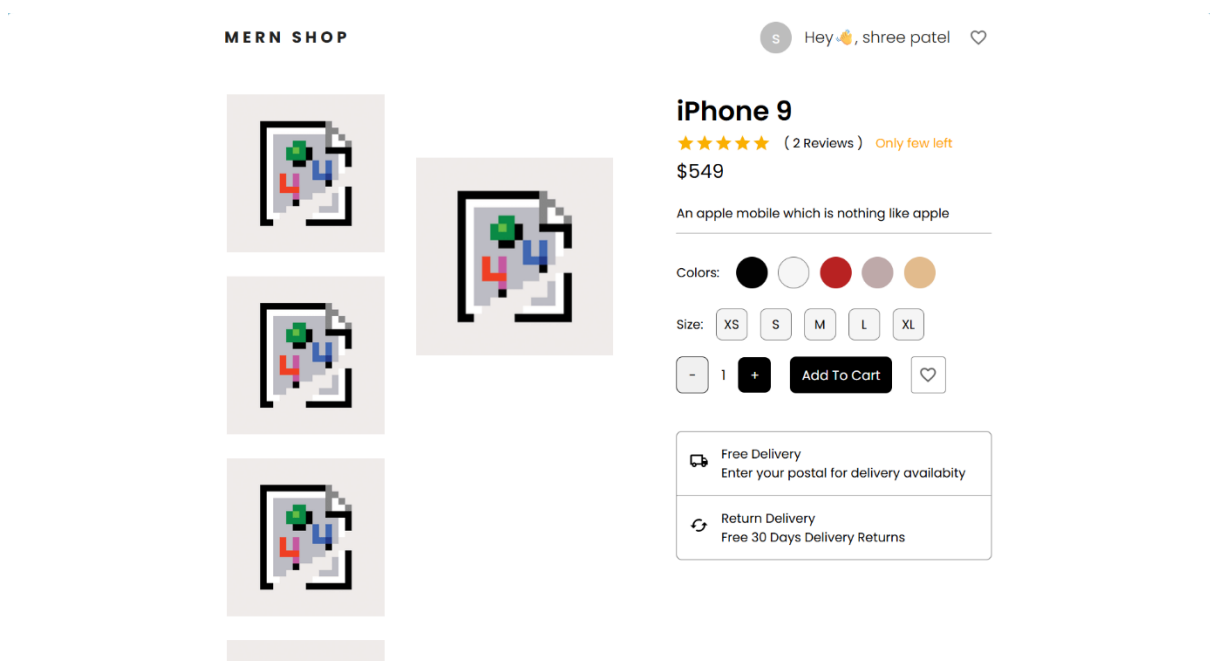


Fig 5.4 Product Interface

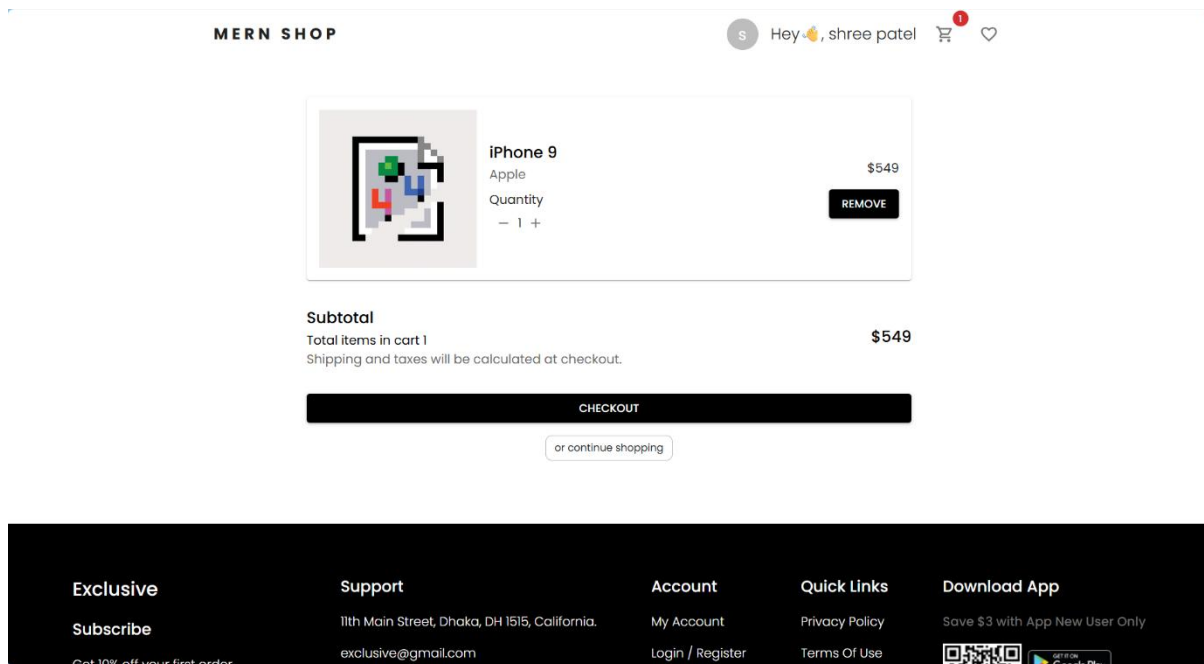


Fig 5.5 Checkout Interface

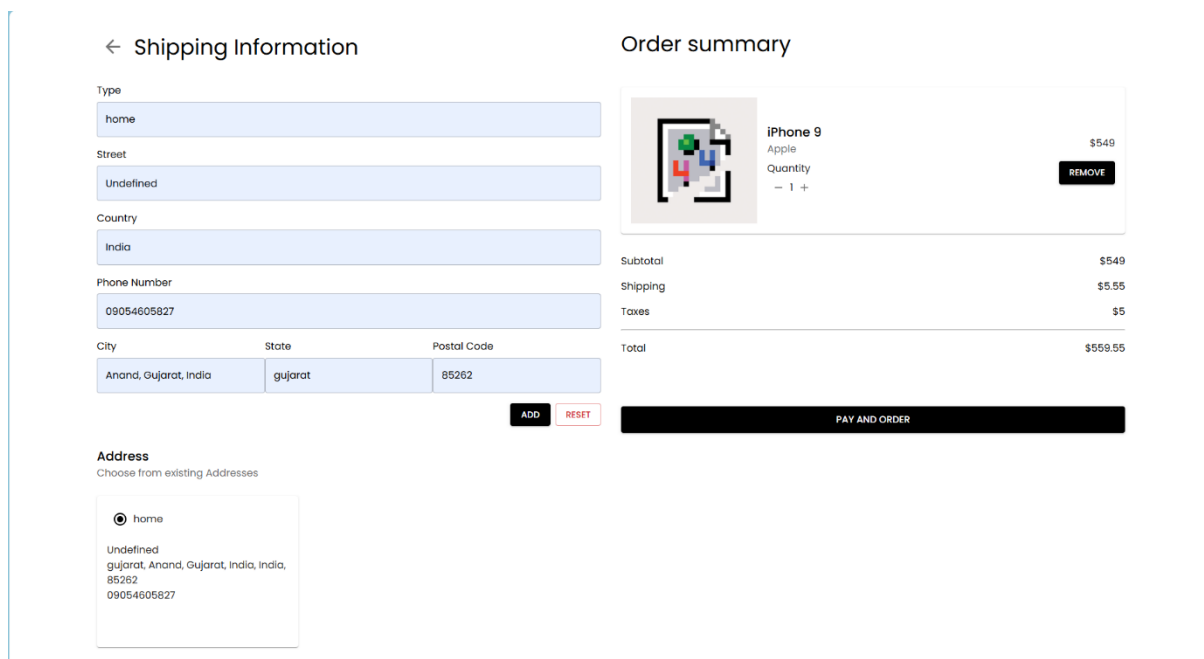


Fig 5.6 Order Summary Interface

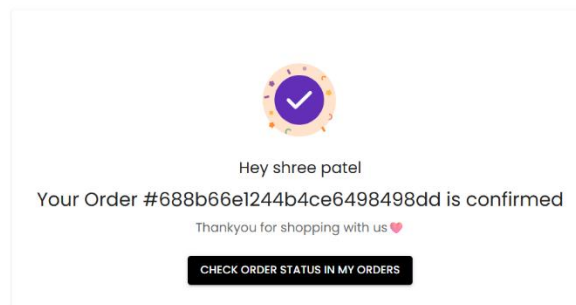


Fig 5.7 Order Completion Interface

MERN SHOP

S Hey 🧡, shree patel ❤️

Order history

Check the status of recent orders, manage returns, and discover similar products.

Order Number	Date Placed	Total Amount	Item: 2
688b6363244b4ce649849864	Thu Jul 31 2025	\$1272.55	
	Fog Scent Xpressio Perfume custom Fog Scent Xpressio Qty: 1 Product details of Best Fog Scent Xpressio Perfume 100ml For Men cool long lasting perfumes for Men	\$13	VIEW PRODUCT BUY AGAIN
	Samsung Universe 9 Samsung Qty: 1 Samsung's new variant which goes beyond Galaxy to the Universe	\$1249	VIEW PRODUCT BUY AGAIN

Status : Pending

Fig 5.8 Order History Interface

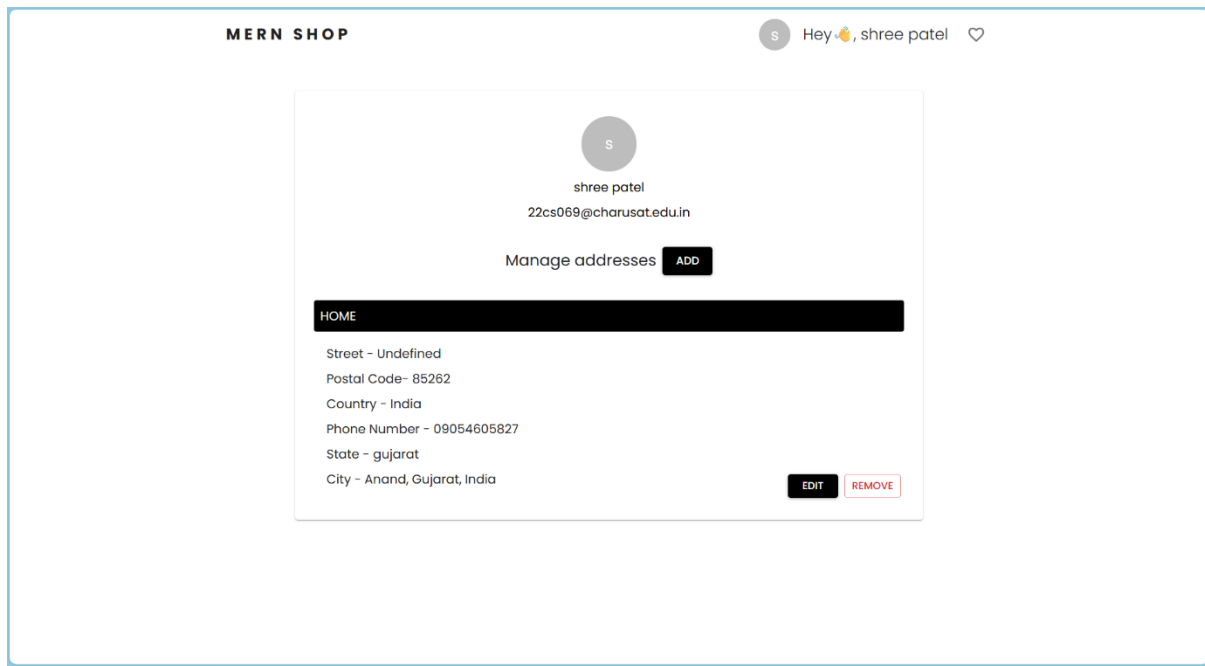


Fig 5.9 Profile Interface

5.1.2 Social-Media Web Application Screenshots

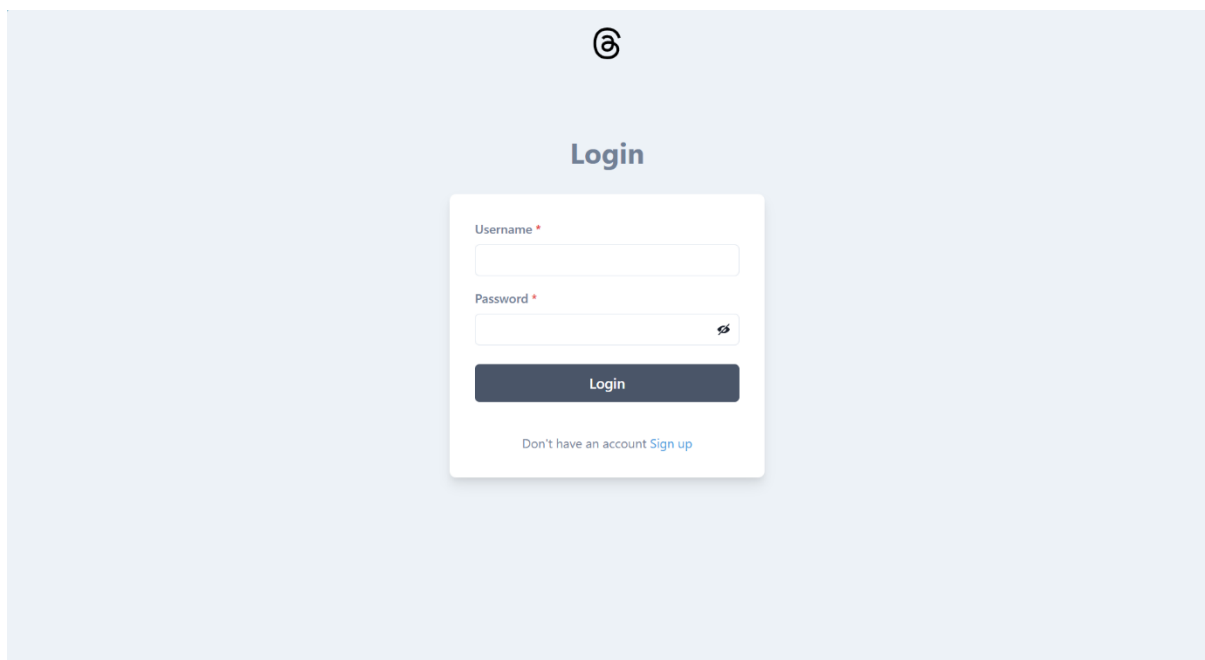
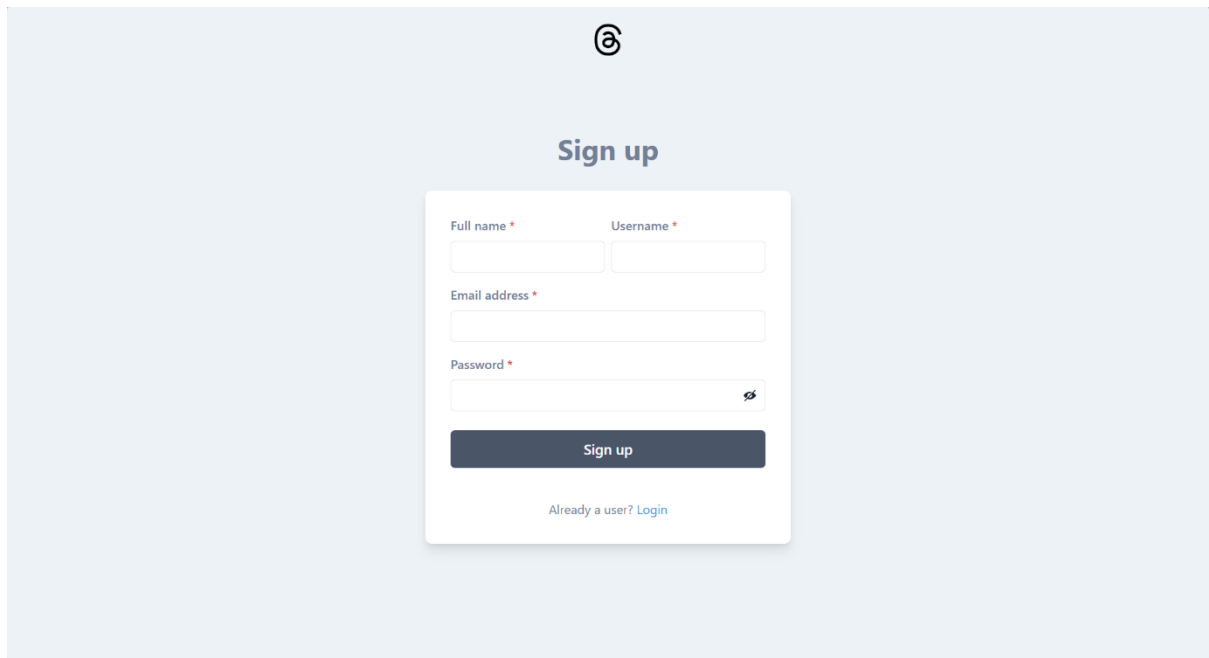


Fig 5.10 login Interface



The screenshot shows a 'Sign up' form centered on a light blue background. At the top center is a circular logo with a stylized '@' symbol. Below it, the text 'Sign up' is displayed in a bold, dark blue font. The form itself is a white card with a subtle shadow. It contains four input fields: 'Full name *', 'Username *', 'Email address *', and 'Password *'. The 'Password' field has a small eye icon to its right. Below the inputs is a dark blue 'Sign up' button. At the bottom of the card, there is a link that says 'Already a user? Login'.

Fig 5.11 SignUp Interface

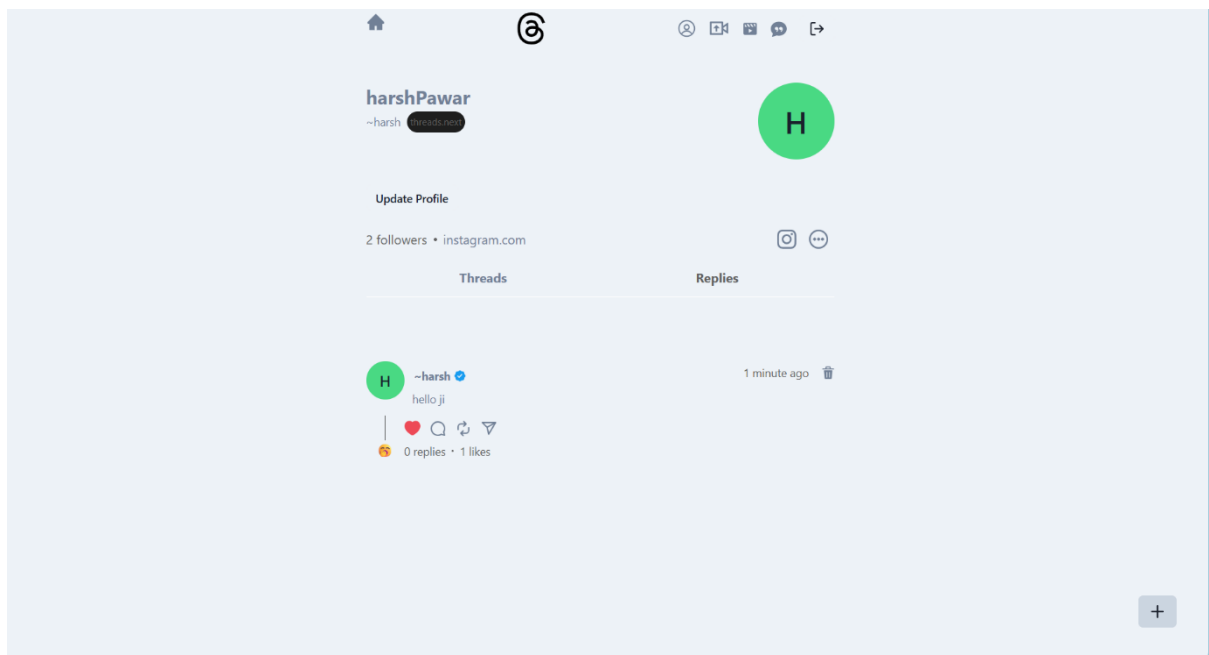


Fig 5.12 User Profile Interface

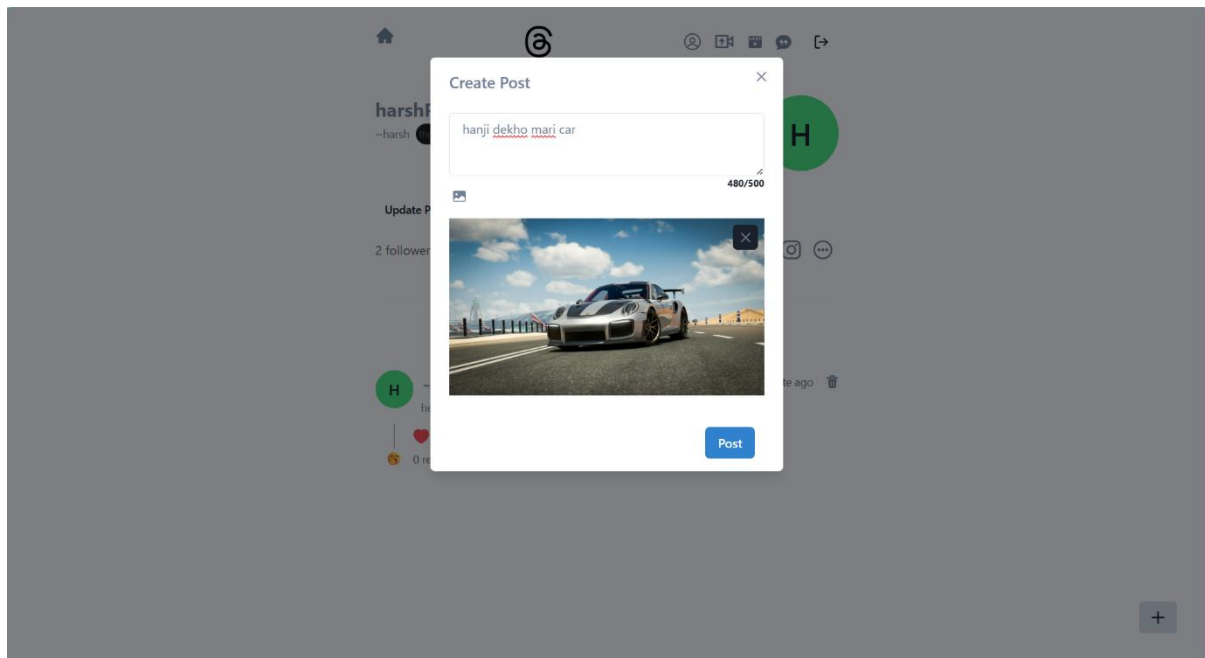


Fig 5.13 Create Post Interface

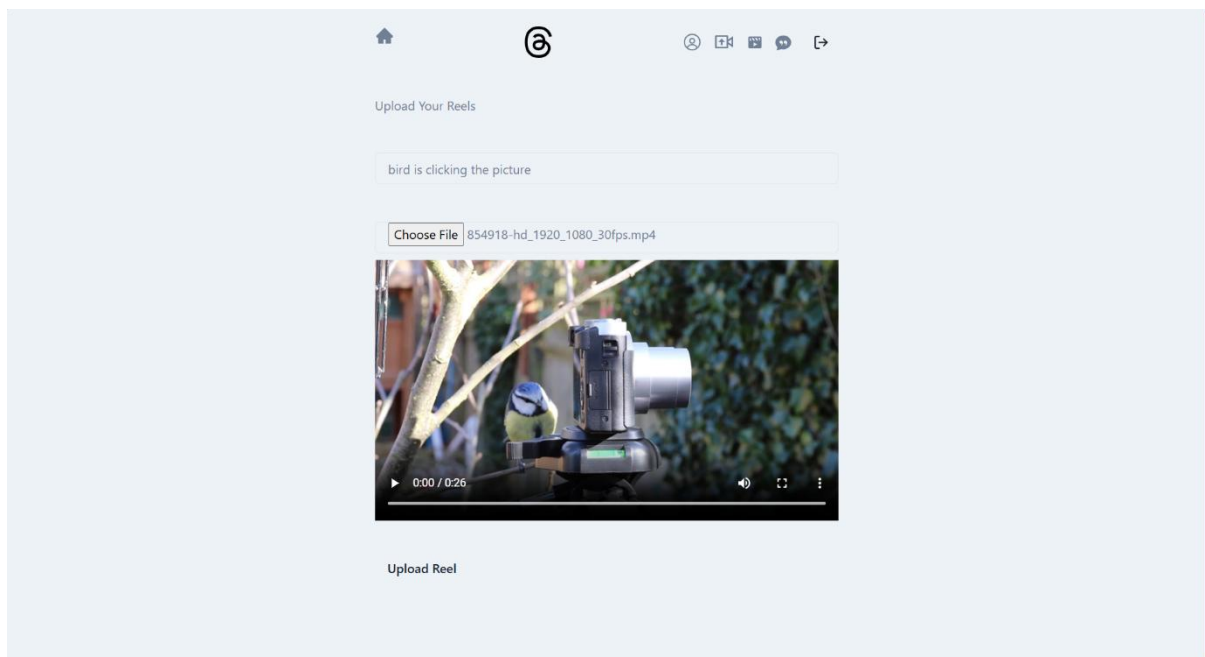


Fig 5.14 Upload Reel Interface

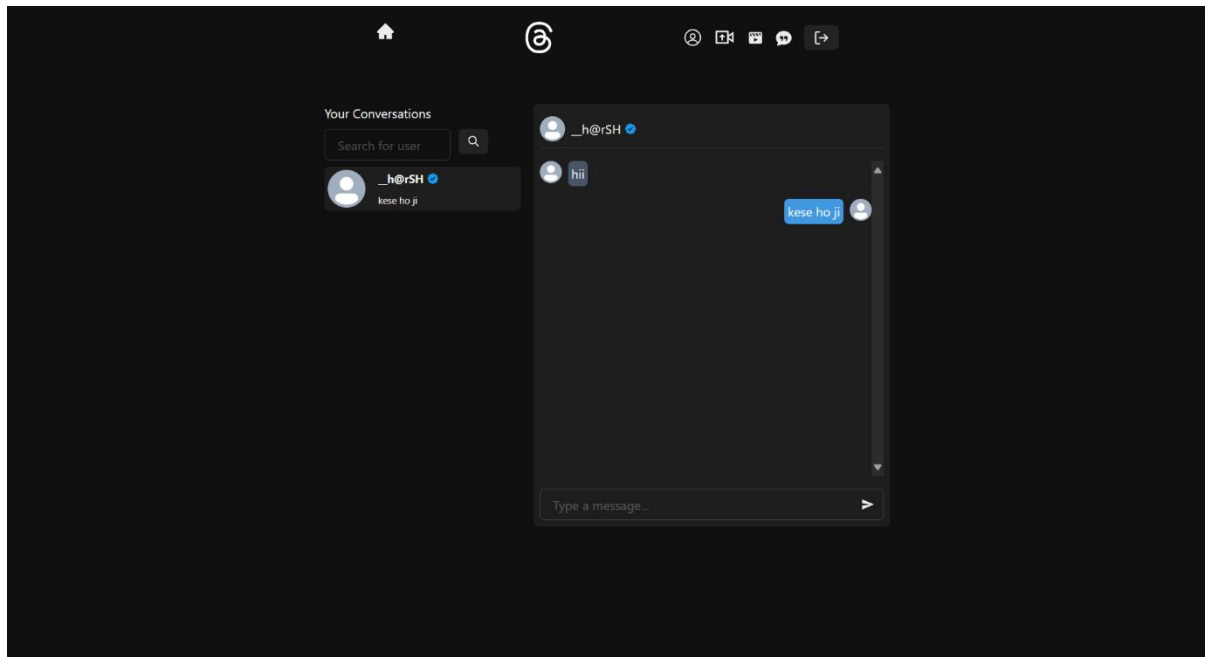


Fig 5.15 Chat Interface

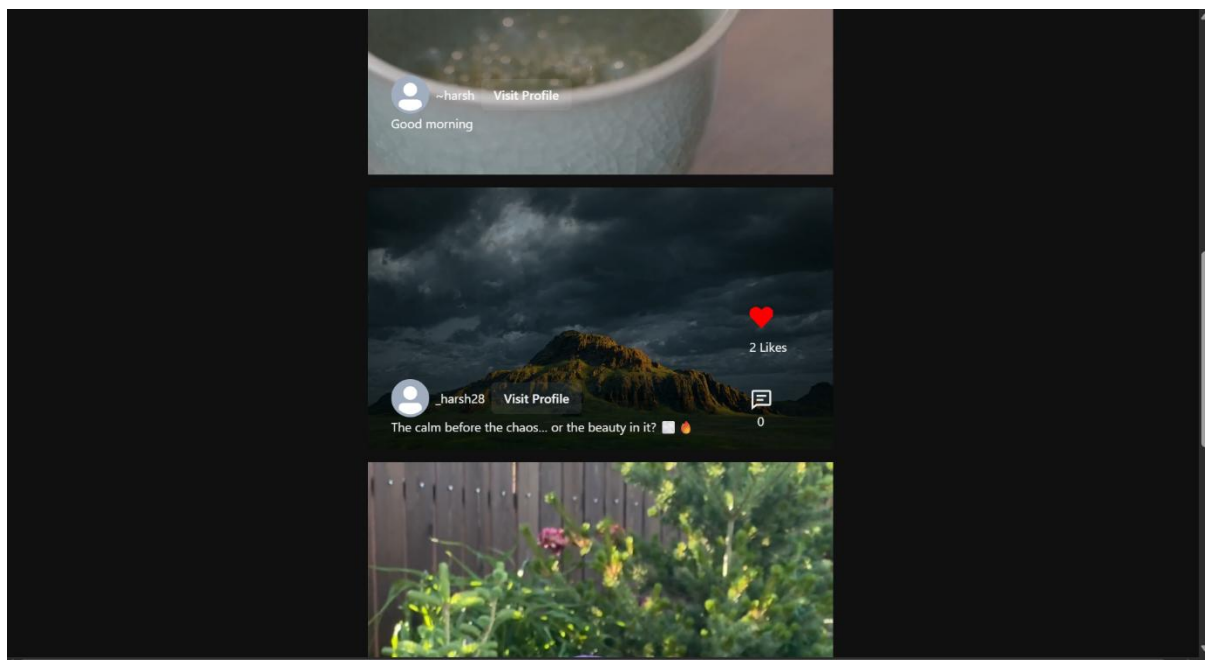


Fig 5.16 Reels Feed Interface

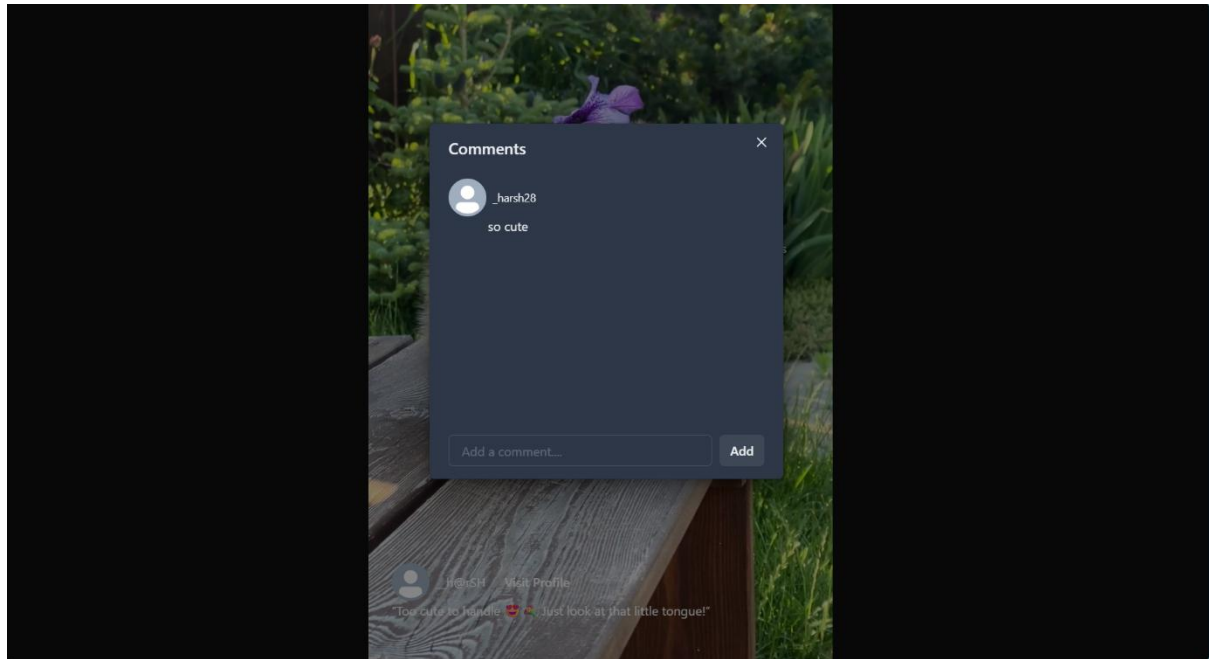


Fig 5.17 Comment Interface

CHAPTER 6

TEST CASES

6.1 E-COMMERCE WEB APPLICATION TEST CASES

Table 6. 1 E-Commerce Web Application Test Cases

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Login	Valid username and password	User registered	Enter username and password, click Login	Valid username, password	Redirect to dashboard	Show dashboard	Pass
Login	Invalid password	User registered	Enter username, wrong password, click Login	Valid username, wrong password	Error message: Invalid credentials		Fail
Registration	New user sign up	None	Fill registration form, submit	New valid details	Successful account creation	Redirect to login	Pass
Registration	Existing email	Email in DB	Fill sign-up with existing email, submit	Existing email	Error: Email already registered		Fail
Product Search	Search by product name	Product exists	Enter product name, click search	Existing product name	Product appears in results		Pass
Add to Cart	Add product to cart	Logged in	Browse catalog, click "Add to	Any product	Product listed in user's cart		Pass

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
			Cart" on a product				
Remove from Cart	Remove product from cart	Product in cart	Open cart, click "Remove" next to a product	Product in cart	Product removed from cart		Pass
Update Quantity	Change product quantity in cart	Product in cart	Change quantity, click "Update"	Various quantities	Correct cart total updates		Pass
Wishlist	Add product to wishlist	Logged in	Click "Add to Wishlist"	Any product	Product visible in wishlist		Pass
Place Order	Checkout flow	Items in cart	Proceed to checkout, enter address, payment details, confirm	Valid details	Order placed successfully	Order history logs	Pass
Payment	Valid card payment	Checking out	Enter valid card info, submit	Valid card	Payment processed, order confirmed		Pass

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Payment	Invalid card payment	Checking out	Enter invalid card info, submit	Invalid card	Error: Payment failed		Fail
Review	Submit product review	Purchased product	Go to order history, select product, write review, submit	Valid review text, stars	Review visible on product page		Pass
Admin Panel	Add new product	Logged as admin	Go to Admin panel, fill product form, submit	Valid product data	Product visible in store/catalog		Pass
Admin Panel	Edit product	Product exists	Edit product details, save changes	Modified fields	Changes visible to users		Pass
Admin Panel	Delete product	Product exists	Delete product	Product ID	Product removed from catalog		Pass
Address Management	Add delivery address	Logged in	Go to profile, add address form, submit	Valid address	Address saved	Address appears	Pass
Order Tracking	View order history	Orders placed	Go to profile/orders	Existing orders	Orders listed with status		Pass

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Logout	Proper session logout	Logged in	Click Logout button	-	User returned to login page	Cookies cleared	Pass

6.2 SOCIAL-MEDIA WEB APPLICATION TEST CASES

Table 6. 2 Social-Media Web Application Test Cases

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Login	Valid credentials	User registered	Enter username and password, click Login	Valid username & password	Login successful, homepage loads	User session active	Pass
Login	Invalid credentials	User exists	Enter username, wrong password, click Login	Valid username, wrong password	Error: Invalid credentials		Fail
Registration	New user registration	None	Fill signup form, submit	Unique username, email	Account created, prompt login		Pass
Registration	Existing username/email	Username/email in DB	Fill signup with existing email, submit	Existing email	Error: User/email already exists		Fail

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Post Creation	Create new post	Logged in	Go to feed, click "Create Post", submit text/image, post	Text, optional image	Post appears in feed/profile		Pass
Like Post	Like a post	Logged in, post exists	Click "Like" on a post	Post ID	Like icon highlights; like count adds		Pass
Unlike Post	Unlike a liked post	Already liked post	Click Like again on liked post	Post ID	Like count decreases by one		Pass
Comment	Add comment to post	Logged in, post exists	Write comment, submit	Comment text	Comment lists under post		Pass
Delete Comment	Remove own comment	Own comment exists	Click Delete on own comment	Comment ID	Comment deleted		Pass
Follow User	Follow another user	User logged in	Visit user profile, click "Follow"	Any other user	Button changes to "Following", follower count increases		Pass
Unfollow User	Unfollow a followed user	Already following	Visit profile, click "Unfollow"	Any followed user	Button reverts, follower count drops		Pass

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Update Profile	Edit profile info or image	Logged in	Navigate to profile, edit details, save	New info/profile picture	Changes reflect on profile page		Pass
Upload Reel	Upload video reel	Logged in	Go to reels, choose file, upload	Short video file	Reel appears in feed/profile		Pass
Chat	Send message to user	Both users registered	Open chat, select user, send message	Text message	Message appears in chat window		Pass
Real-Time Chat	Live receipt of messages	Chat open with user	Second user replies while first is viewing chat	Message from another user	Message appears instantly		Pass
Search Users	Use search bar to find users	Registered users exist	Enter existing username in search	Any part of username	Matching users displayed		Pass
Notifications	Like/comment/follow triggers notif	Trigger event	Perform like, comment, or follow for user	Action on user content/profile	Receiver sees notification icon/update		Pass
Session Expiry	Token/session expiration	Valid session expired	Wait until session expires, refresh	-	User asked to log in again		Pass

Test Case Scenario	Test Case Name	Pre-condition	Testing Steps	Testing Data	Expected Result	Post Condition	Status (Pass/Fail)
Logout	Log out from profile	Logged in	Click Logout	-	Returned to login, session ends		Pass

CHAPTER 7

LIMITATIONS AND FUTURE WORK

7.1 E-COMMERCE WEB APPLICATION

7.1.1 Limitations

- **Payment Integration:** The application currently simulates order placement and payment without integration to live payment gateways (such as Stripe or Razorpay). This limits deployment for real-world commerce where actual transactions are necessary.
- **Product Recommendation:** No machine learning or user-based recommendation engine is in place. All users see the same product listings and offers, which can affect personalization and conversion rates.
- **Email and Notification Services:** OTP and password reset flows are set up, but comprehensive transactional email notifications (such as order dispatch, delivery updates, etc.) and in-app notifications are minimal.
- **Inventory Management:** Stock depletion and dynamic inventory tracking are basic. There is no threshold alert system for low inventory or automated restocking notifications.
- **Scalability Constraints:** The backend and database were designed for medium-load use cases and may require upgrades in connection pooling, caching, and sharding for large-scale, high-concurrency deployments.
- **Limited Mobile Optimization:** While responsive, the UI is not fully optimized for mobile-first user experiences or progressive web app (PWA) deployment.

7.1.2 Future Work

- **Payment Gateway Integration:** Integrating real payment providers and supporting multiple payment methods, including digital wallets and UPI.
- **Personalized Recommendations:** Implementing collaborative filtering or content-based recommendation systems.
- **Advanced Inventory Management:** Adding real-time stock tracking, supplier management, and analytics dashboards.

- **Comprehensive Notifications:** Building an in-app and email notification system for every user transaction stage.
- **PWA and Mobile App:** Extending the platform as a PWA or developing native Android/iOS applications using React Native.
- **Multi-language/Region Support:** Localizing the application for different languages and currencies to reach global users.
- **Automated Testing and CI/CD:** Expanding test coverage and integrating CI/CD pipelines for continuous deployment and quality assurance.

7.2 SOCIAL-MEDIA WEB APPLICATION

7.2.1 Limitations

- **Push Notifications:** Real-time features like chat use Socket.io, but mobile/web push notifications for offline users are not yet implemented.
- **Media Optimization:** Image and video uploads rely on Cloudinary, with basic optimization. There is no user-side control over compression, file size limits, or offline uploads.
- **Moderation and Reporting:** There is minimal content moderation. No automated system for detecting hateful content, spam, or abuse, and reporting functionality for users is basic.
- **Scalability of Chat and Reels:** While the real-time chat and reels upload systems work well for small groups, scaling to support thousands of concurrent users requires more robust message brokering and CDN integration for video streaming.
- **Privacy Controls:** User controls for content privacy (private/public posts, story controls, etc.) are limited.
- **Analytics:** There is a lack of user analytics, such as post reach, engagement insights, or chat activity statistics for end-users and administrators.

7.2.2 Future Work

- **Push Notification Integration:** Implementing Firebase Cloud Messaging or a similar service to support web and mobile notifications for messages, likes, comments, and follows.
- **Media Handling Enhancements:** Adding bulk upload, scheduled posts, advanced compression, and offline-friendly capabilities.
- **Automated Moderation:** Leveraging AI and machine learning to detect and filter out spam, explicit content, and abusive language, with detailed reporting and appeal systems.
- **Horizontal Scaling:** Adopting microservice-based architecture for chats and media, introducing load balancers, and optimizing message delivery for high concurrency.
- **Enhanced Privacy Features:** Supporting private accounts, restricted posts, customizable follower lists, and story/reels privacy.
- **User and Admin Analytics Dashboards:** Building dashboards for engagement tracking, growth metrics, and platform usage trends.
- **Third-Party Integrations:** Allowing social sign-in (Google, Facebook), cross-platform posting, and sharing capabilities.

CHAPTER 8

CONCLUSION

8.1 CONCLUSION

During my internship at CodeAlpha, I had the valuable opportunity to design and develop two complete web applications—an E-commerce platform and a Social Media app—using the MERN stack (MongoDB, Express.js, React, Node.js). This hands-on experience helped me strengthen my skills in secure authentication, API integration, state management, and responsive UI design, while also giving me exposure to real-time data handling, cloud-based media storage, and scalable architecture. Working on the E-commerce app taught me how to implement core features like user management, inventory control, shopping carts, order processing, and simulated secure payments with a modern, user-friendly interface. On the other hand, the Social Media app deepened my understanding of building real-time features like chat with Socket.io, post management, notifications, and dynamic user interactions. I also gained experience with tools like Redux Toolkit, Material-UI, Cloudinary, and followed collaborative practices such as Git version control and API documentation. From solving backend challenges like API security to tackling frontend performance issues, every part of this internship helped me grow as a full-stack developer. Overall, this experience bridged the gap between theory and practice, reinforced my passion for software engineering, and laid a strong foundation for my future career.

REFERENCES

1. MongoDB: <https://www.mongodb.com/docs/>
2. Express.js: <https://expressjs.com/>
3. React: <https://reactjs.org/docs/getting-started.html>
4. Redux Toolkit: <https://redux-toolkit.js.org/>
5. Material-UI: <https://mui.com/>
6. Socket.io: <https://socket.io/docs/v4/>
7. Cloudinary: <https://cloudinary.com/documentation>
8. Multer: <https://github.com/expressjs/multer>
9. MDN Web Docs: <https://developer.mozilla.org/>
10. Stack Overflow :<https://stackoverflow.com/>
11. npm documentation: <https://docs.npmjs.com/>
12. Postman docs: <https://learning.postman.com/>